ESTACION METEOROLOGICA

ESP32

TFT ST7789V 320x240 LANDSCAPE

DHT11

PWA

Ariel Izquierdo Ereza

2026

Ante todo aclarar que no soy especialista ni programador, mi única experiencia es en algunos proyectos sencillos con ARDUINO NANO, éste es mi primer proyecto con ESP-32 que hasta hace muy poco no sabía ni que existían. La idea de la Estación Meteorológica es un bichito que desde hace tiempo rondaba por mi cabeza. Mi primer intento fue presisamente con un Arduino NANO y un módulo wifi extraido de un termostato averiado, pero la poca capacidad me llevó a buscar otras alternativas, ya que a pesar de que funcionaba, no estaba a la altura de lo que esperaba y enseguida deseché la idea.

Esta versión se divide en tres partes fundamentales a nivel de Hardware, una el ESP-32 que en este caso utilicé una nodeMCU-32S, una pantalla LCD S7789 de 2 pulgadas 320x240 y un Sensor de interior, que en mi caso utilicé un DHT11 por ser el que tenía en casa. También contiene un pulsador de utilitario para el avance de las pantallas manualmente y un LED que indica el estado de la conexión.

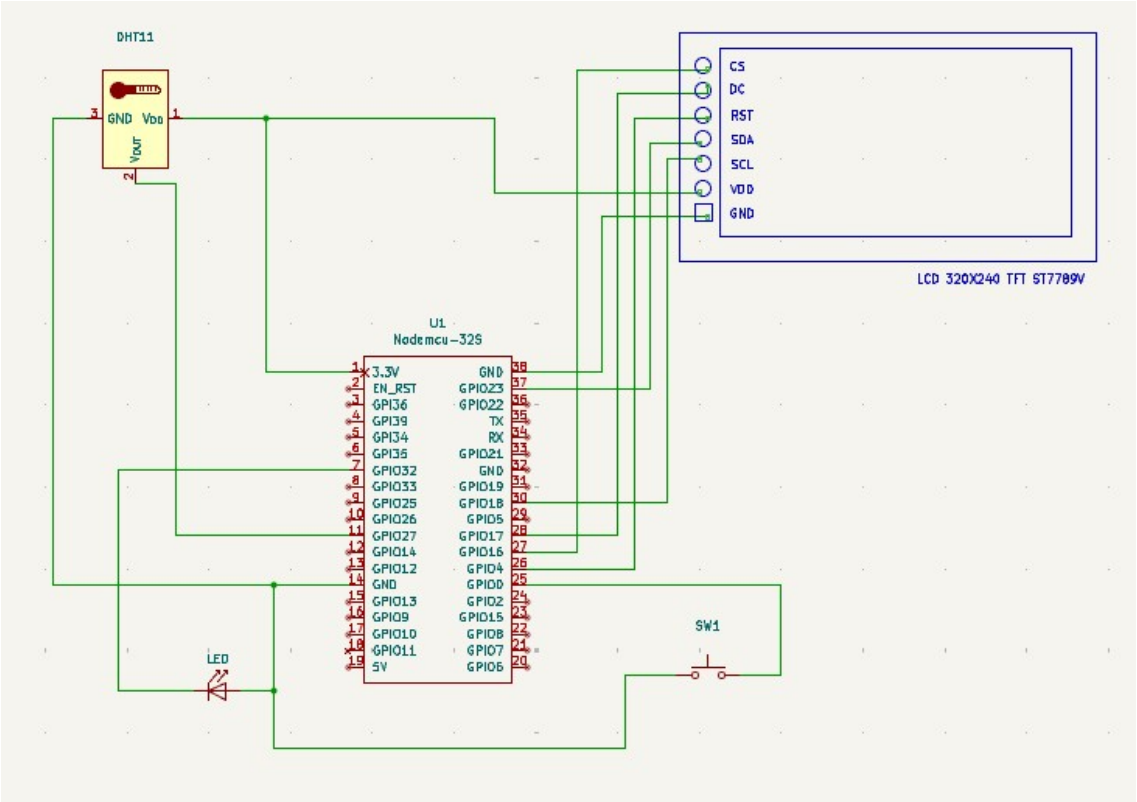


Ilustración 1

En la Ilustracón 1, podemos ver el esquema de conexión de todos los componentes utilizados.

El Código que expongo a continuación, se va explicando por sí sólo, ya que está bastante comentado en todas sus secciones:

```
/* ==================================================================
 *  ESTACION MET — ESP32 + TFT ST7789V 320x240 LANDSCAPE + PWA
 *  Orientacion: LANDSCAPE (320 ancho x 240 alto)
 *
 *  CONEXION ST7789V (SPI Hardware ESP32):
 *    GND  -> GND
 *    VCC  -> 3.3V
 *    SCL  -> GPIO 18 (SCLK)
 *    SDA  -> GPIO 23 (MOSI)
 *    RES  -> GPIO 4  (RESET)
 *    DC   -> GPIO 17 (Data/Command)
 *    CS   -> GPIO 16 (Chip Select)
 *    BL   -> 3.3V (retroiluminacion siempre encendida)
 *
 *  LIBRERIAS NECESARIAS (instalar en Arduino IDE):
 *    - Adafruit GFX Library
 *    - Adafruit ST7735 and ST7789 Library
 *    - ArduinoJson
 *    - DHT sensor library (Adafruit)
```

```
 *
 *  5 PANTALLAS ROTATIVAS (15s cada una):
 *    0 - Reloj + mini temps
 *    1 - Interior (DHT11)
 *    2 - Exterior (OWM)
 *    3 - Detalles (presion, viento, humedad)
 *    4 - Prevision 3 dias
 *
 *  CAMBIOS v6:
 *    - Checkbox "Hora de invierno" en pantalla Config de la PWA
 *    - cfg_winterTime guardado en NVS (clave "winterTime")
 *    - Si activo: TZ = CET-1 (UTC+1 fijo), si inactivo: CET-1CEST (UTC+2 verano)
 *    - winterTime expuesto en /api/data y aceptado en /api/config
 *    - Toggle switch visual en la PWA, se rellena automaticamente al abrir Config
 *
 *  CAMBIOS v5:
 *    - Offset de calibracion temperatura DHT11 (cfg_tOffset)
 *    - Se guarda en NVS y se ajusta desde la PWA sin recompilar
 *    - Corregidos bugs: fetchOWM no sobreescribe offset,
 *      JS saveConfig incluye tOffset, updateUI rellena el campo,
 *      eliminado codigo suelto en loadHistory, typo en handleData
 *
 *  API REST:
 *    GET  /          -> PWA HTML
 *    GET  /api/data    -> JSON actual (incluye tOffset)
 *    GET  /api/history -> JSON historial 24h
 *    POST /api/login   -> { "user":"...", "pass":"..." }
 *    POST /api/config  -> { token, city, country, ssid, pass, tOffset, winterTime }
 *    POST /api/reboot  -> reiniciar ESP32
 * ==================================================================
 */

#include <WiFi.h>
#include <HTTPClient.h>
#include <ArduinoJson.h>
#include <Wire.h>
#include <time.h>
#include <DHT.h>
#include <WebServer.h>
#include <Preferences.h>
#include <SPI.h>
#include <Adafruit_GFX.h>
#include <Adafruit_ST7789.h>

// ── Pines TFT ST7789V ──────────────────────────────────────
#define TFT_CS   16
#define TFT_DC   17
#define TFT_RST  4
#define BTN_PIN  0
// SCLK -> GPIO18, MOSI -> GPIO23 (SPI hardware ESP32)

// ── Dimensiones en LANDSCAPE ───────────────────────────────
#define TFT_W   320
#define TFT_H   240
```

```
// ── Colores RGB565 ──────────────────────────────────────────────
#define C_BG      0x0823
#define C_PANEL   0x0C46
#define C_CARD    0x1069
#define C_CARD2   0x0C4A
#define C_ACCENT  0x065F
#define C_GREEN   0x3666
#define C_YELLOW  0xFEA0
#define C_ORANGE  0xFB00
#define C_RED     0xF800
#define C_WHITE   0xFFFF
#define C_LGRAY   0xAD75
#define C_MUTED   0x52AA
#define C_BLUE    0x031F
#define C_PURPLE  0x781F
#define C_BORDER  0x18C3

// ── Configuracion por defecto ──────────────────────────────────
#define DEF_SSID    "SU WIFFI"
#define DEF_PASS    "SU PASSWORD"
#define DEF_CITY    "SU CIUDAD"
#define DEF_COUNTRY "ES"  //PAIS CODIGO UNIVERSAL

// ── Credenciales acceso Config ─────────────────────────────────
#define CFG_USER    "SU USUARIO"     //PARA ACCEDER PAGINA CONFIGURACION
#define CFG_PASS    "SU CONTRASEÑA"  // PAW HTML

const char* OWM_KEY = "SU API DE OPENWEATHER";
const char* TZ_STR  = "SU ZONA HORARIA";  / EJEMP: ZONA HORARIA MADRID:  CET-
1CEST,M3.5.0,M10.5.0/3

#define LED_STATUS 32
#define DHT_PIN    27
#define DHT_TYPE   DHT11

#define NUM_SCREENS  5
#define SCR_CYCLE  15000UL   //20000 TIEMPO DE CAMBIO DE PANTALLAS
#define T_OWM     300000UL   // TIEMPO DE CONSULTA EN OPENWEATHER
#define T_DHT       5000UL   //5000 TIEMPO DE CONSULTA AL DHT11
#define T_HISTORY 150000UL   // TIEMPO DE GUARDADO DE LA HISTORIA

// ── Historial circular 24h ─────────────────────────────────────
#define HIST_SIZE 288
struct HistSample { uint32_t ts; float tExt, tInt; int hum; float pressure; };
HistSample hist[HIST_SIZE];
int histHead=0, histCount=0;

// ── Objetos principales ────────────────────────────────────────
Adafruit_ST7789 tft = Adafruit_ST7789(TFT_CS, TFT_DC, TFT_RST);
DHT        dht(DHT_PIN, DHT_TYPE);
WebServer   server(80);
Preferences prefs;
```

```
char  cfg_ssid[32], cfg_pass[64], cfg_city[32], cfg_country[8];
float cfg_tOffset    = 0.0f;   // offset calibracion temperatura interior
bool  cfg_winterTime = false;  // true = hora de invierno (UTC+1), false = verano (UTC+2)

struct FcDay { char fecha[6]; char diaSem[5]; int tMax, tMin, wid; };
struct Wx {
  char  city[20]  = "";
  float tExt      = 0;
  float pressure  = 1013;
  float tInt      = 0;
  float hInt      = 0;
  int   hum       = 0;
  int   wind      = 0;
  int   windDeg   = 0;
  int   wid       = 800;
  char  desc[32]  = "";
  bool  owmOk     = false;
  bool  dhtOk     = false;
  bool  wifiOk    = false;
  FcDay fc[3];
  int   todayMax  = 0;
  int   todayMin  = 0;
  int   todayWid  = 800;
} W;

uint8_t  curScreen  = 0;
uint8_t  lastScreen = 255;
unsigned long tOwm=0, tDht=0, tScr=0, tHistory=0;



//═══════════════════════════════════════════════════════════════════════════
//  UTILIDADES
//═══════════════════════════════════════════════════════════════════════════
const char* degToDir(int deg) {
  const char* d[] = {"N","NE","E","SE","S","SO","O","NO"};
  return d[((deg+22)%360)/45];
}

void loadConfig() {
  prefs.begin("estmet", true);
  strlcpy(cfg_ssid,    prefs.getString("ssid",    DEF_SSID).c_str(),    32);
  strlcpy(cfg_pass,    prefs.getString("pass",    DEF_PASS).c_str(),    64);
  strlcpy(cfg_city,    prefs.getString("city",    DEF_CITY).c_str(),    32);
  strlcpy(cfg_country, prefs.getString("country", DEF_COUNTRY).c_str(), 8);
  cfg_tOffset    = prefs.getFloat("tOffset",    0.0f);
  cfg_winterTime = prefs.getBool("winterTime",  false);
  prefs.end();
}

void saveConfig() {
  prefs.begin("estmet", false);
  prefs.putString("ssid",    cfg_ssid);
  prefs.putString("pass",    cfg_pass);
```

```
  prefs.putString("city",    cfg_city);
  prefs.putString("country", cfg_country);
  prefs.putFloat("tOffset",   cfg_tOffset);
  prefs.putBool("winterTime", cfg_winterTime);
  prefs.end();
}

void addHistory() {
  struct tm ti;
  if (!getLocalTime(&ti)) return;
  time_t now; time(&now);
  hist[histHead] = {(uint32_t)now, W.tExt, W.tInt, W.hum, W.pressure};
  histHead = (histHead+1) % HIST_SIZE;
  if (histCount < HIST_SIZE) histCount++;
}

//═══════════════════════════════════════════════════════════════════════════
//  HELPERS DE DIBUJO TFT
//═══════════════════════════════════════════════════════════════════════════
void printCenter(const char* s, int y, uint16_t col, uint8_t sz,
              int x0=0, int wide=TFT_W) {
  tft.setTextSize(sz);
  tft.setTextColor(col, C_BG);
  int16_t bx, by; uint16_t bw, bh;
  tft.getTextBounds(s, 0, 0, &bx, &by, &bw, &bh);
  tft.setCursor(x0 + (wide - (int)bw) / 2, y);
  tft.print(s);
}
void printCenterBg(const char* s, int y, uint16_t col, uint16_t bg, uint8_t sz,
                int x0=0, int wide=TFT_W) {
  tft.setTextSize(sz);
  tft.setTextColor(col, bg);
  int16_t bx, by; uint16_t bw, bh;
  tft.getTextBounds(s, 0, 0, &bx, &by, &bw, &bh);
  tft.setCursor(x0 + (wide - (int)bw) / 2, y);
  tft.print(s);
}
void drawCard(int x, int y, int w, int h,
            uint16_t fill=C_CARD, uint16_t border=C_BORDER) {
  tft.fillRoundRect(x, y, w, h, 6, fill);
  tft.drawRoundRect(x, y, w, h, 6, border);
}
void statusDot(int cx, int cy, bool on,
             uint16_t cOn=C_GREEN, uint16_t cOff=C_RED) {
  tft.fillCircle(cx, cy, 5, on ? cOn : cOff);
  tft.drawCircle(cx, cy, 5, C_BORDER);
}

//═══════════════════════════════════════════════════════════════════════════
//  SIMBOLO DE GRADO — primitiva grafica, independiente de codepage
//═══════════════════════════════════════════════════════════════════════════
void printDegC(int x, int y, uint16_t col, uint16_t bg, uint8_t sz) {
  int r  = sz * 2 + 1;
  int cx = x + r + 1;
```

```
  int cy = y + r;
  tft.fillCircle(cx, cy, r, bg);
  tft.drawCircle(cx, cy, r, col);
  tft.setTextSize(sz);
  tft.setTextColor(col, bg);
  tft.setCursor(x + r*2 + 5, y + (sz == 1 ? 1 : 0));
  tft.print("C");
}

void printDeg(int x, int y, uint16_t col, uint16_t bg, uint8_t sz) {
  int r  = sz * 2;
  int cx = x + r;
  int cy = y + r;
  tft.fillCircle(cx, cy, r, bg);
  tft.drawCircle(cx, cy, r, col);
}

//═══════════════════════════════════════════════════════════════════════════
//  CONVERSION UTF-8 → CP437 para ñ y tildes
//═══════════════════════════════════════════════════════════════════════════
String toCP437(const char* s) {
  String out = "";
  uint8_t* p = (uint8_t*)s;
  while (*p) {
    if (*p == 0xC3) {
      p++;
      switch (*p) {
        case 0xB1: out += (char)0xA4; break; // ñ
        case 0xA1: out += (char)0xA0; break; // á
        case 0xA9: out += (char)0x82; break; // é
        case 0xAD: out += (char)0xA1; break; // í
        case 0xB3: out += (char)0xA2; break; // ó
        case 0xBA: out += (char)0xA3; break; // ú
        case 0x91: out += (char)0xA0; break; // Á
        case 0x89: out += (char)0x90; break; // É
        case 0x93: out += (char)0xA2; break; // Ó
        default:   out += '?';       break;
      }
    } else {
      out += (char)(*p);
    }
    p++;
  }
  return out;
}

//═══════════════════════════════════════════════════════════════════════════
//  CLASIFICACION ICONO POR WID
//═══════════════════════════════════════════════════════════════════════════
char widType(int wid) {
  if (wid >= 200 && wid < 300) return 't';
  if (wid >= 300 && wid < 400) return 'd';
  if (wid >= 400 && wid < 600) return 'r';
  if (wid >= 600 && wid < 700) return 'n';
```

```
  if (wid >= 700 && wid < 800) return 'f';
  if (wid == 800)            return 's';
  if (wid == 801)            return 'p';
  if (wid == 802)            return 'm';
  if (wid == 803)            return 'c';
  if (wid == 804)            return 'c';
  return 'c';
}

//═══════════════════════════════════════════════════════════════════════════
//  ICONOS METEOROLOGICOS CON PRIMITIVAS GFX
//═══════════════════════════════════════════════════════════════════════════
void drawIcon(int cx, int cy, char t, int r) {
  switch (t) {
    case 's':
      tft.fillCircle(cx, cy, r, C_YELLOW);
      for (int a = 0; a < 360; a += 45) {
        float rd = a * PI / 180.f;
        tft.drawLine(cx+(r+2)*cos(rd), cy+(r+2)*sin(rd),
                     cx+(r+r/2)*cos(rd), cy+(r+r/2)*sin(rd), C_YELLOW);
      }
      break;
    case 'p':
      tft.fillCircle(cx-r/3, cy+r/4, r*3/4, C_YELLOW);
      for (int a = 0; a < 360; a += 60) {
        float rd = a * PI / 180.f;
        tft.drawLine(cx-r/3+(r*3/4+2)*cos(rd), cy+r/4+(r*3/4+2)*sin(rd),
                     cx-r/3+(r*3/4+r/3)*cos(rd), cy+r/4+(r*3/4+r/3)*sin(rd), C_YELLOW);
      }
      tft.fillRoundRect(cx-r/2, cy-r/4, r*3/2, r/2, r/4, 0xAD55);
      tft.fillCircle(cx,     cy-r/4, r*2/5, 0xAD55);
      tft.fillCircle(cx+r/3, cy-r/3, r/2,   0xAD55);
      break;
    case 'm':
      tft.fillCircle(cx-r/2, cy+r/3, r*2/3, C_YELLOW);
      tft.fillRoundRect(cx-r, cy-r/4, r*2, r*2/3, r/3, 0xAD55);
      tft.fillCircle(cx-r/3, cy-r/4, r*2/5, 0xAD55);
      tft.fillCircle(cx+r/4, cy-r/3, r/2,   0xAD55);
      break;
    case 'c':
      tft.fillRoundRect(cx-r, cy-r/5, r*2, r*3/4, r/3, 0x8C71);
      tft.fillCircle(cx-r/3, cy-r/5, r*2/5, 0x8C71);
      tft.fillCircle(cx+r/4, cy-r/3, r/2,   0x8C71);
      tft.fillCircle(cx-r*2/3, cy, r/4, 0x7BEF);
      break;
    case 'd':
      tft.fillRoundRect(cx-r, cy-r/5, r*2, r*2/3, r/3, 0x8C71);
      tft.fillCircle(cx-r/3, cy-r/5, r*2/5, 0x8C71);
      tft.fillCircle(cx+r/4, cy-r/3, r/2,   0x8C71);
      for (int i = -1; i <= 1; i++) {
        int gx = cx + i*(r/2);
        tft.drawLine(gx, cy+r/2, gx-2, cy+r, C_ACCENT);
      }
      break;
```

```
    case 'r':
      tft.fillRoundRect(cx-r, cy-r/5, r*2, r*2/3, r/3, 0x7BEF);
      tft.fillCircle(cx-r/3, cy-r/5, r*2/5, 0x7BEF);
      tft.fillCircle(cx+r/4, cy-r/3, r/2,   0x7BEF);
      for (int i = -1; i <= 1; i++) {
        int gx = cx + i*(r/2);
        tft.drawLine(gx, cy+r/2, gx-4, cy+r+4, C_ACCENT);
        tft.fillCircle(gx-4, cy+r+4, 2, C_ACCENT);
      }
      break;
    case 'n':
      tft.fillRoundRect(cx-r, cy-r/5, r*2, r*2/3, r/3, 0x7BEF);
      tft.fillCircle(cx-r/3, cy-r/5, r*2/5, 0x7BEF);
      tft.fillCircle(cx+r/4, cy-r/3, r/2,   0x7BEF);
      for (int i = -1; i <= 1; i++)
        tft.fillCircle(cx+i*(r/2), cy+r*3/4, 3, C_WHITE);
      break;
    case 'f':
      for (int j = 0; j < 3; j++)
        tft.fillRoundRect(cx-r, cy-r/3+j*(r/2), r*2, r/4, r/8, 0x8C71);
      break;
    case 't':
      tft.fillRoundRect(cx-r, cy-r/5, r*2, r*2/3, r/3, 0x4228);
      tft.fillCircle(cx-r/3, cy-r/5, r*2/5, 0x4228);
      tft.fillCircle(cx+r/4, cy-r/3, r/2,   0x4228);
      tft.fillTriangle(cx, cy+r/4, cx-r/3, cy+r, cx+r/6, cy+r*3/5, C_YELLOW);
      tft.fillTriangle(cx+r/6, cy+r*3/5, cx-r/6, cy+r, cx+r/2, cy+r, C_YELLOW);
      break;
  }
}

//═══════════════════════════════════════════════════════════════════════════
//  HEADER COMUN
//═══════════════════════════════════════════════════════════════════════════
#define HDR_H  38
#define BODY_Y 38
#define BODY_H (TFT_H - BODY_Y)

void drawHeader() {
  tft.fillRect(0, 0, TFT_W, HDR_H, C_PANEL);
  tft.drawFastHLine(0, HDR_H-1, TFT_W, C_BORDER);

  tft.setTextSize(2);
  tft.setTextColor(C_ACCENT, C_PANEL);
  tft.setCursor(8, 9);
  tft.print(toCP437(cfg_city));

  int sp = 18;
  int sx = TFT_W/2 - (NUM_SCREENS-1)*sp/2 + 15;
  for (int i = 0; i < NUM_SCREENS; i++) {
    int dx = sx + i*sp;
    if (i == curScreen) {
      tft.fillCircle(dx, 21, 5, C_ACCENT);
      tft.drawCircle(dx, 21, 5, C_WHITE);
```

```
    } else {
      tft.fillCircle(dx, 21, 4, C_BG);
      tft.drawCircle(dx, 21, 4, C_MUTED);
    }
  }

  tft.setTextSize(1);
  tft.setTextColor(C_ACCENT, C_PANEL);
  tft.setCursor(220, 5);  tft.print("WiFi");
  statusDot(249, 10, W.wifiOk);
  tft.setCursor(264, 5);  tft.print("OWM");
  statusDot(291, 10, W.owmOk);
  tft.setCursor(224, 22); tft.print("DHT");
  statusDot(249, 27, W.dhtOk, C_GREEN, C_MUTED);

  struct tm ti;
  if (getLocalTime(&ti, 0)) {
    tft.setTextSize(2);
    char tb[6]; snprintf(tb, 6, "%02d:%02d", ti.tm_hour, ti.tm_min);
    tft.setTextColor(C_LGRAY, C_PANEL);
    tft.setCursor(260, 20); tft.print(tb);
  }
}
//═══════════════════════════════════════════════════════════════════════════
//  PANTALLA 0 — RELOJ
//═══════════════════════════════════════════════════════════════════════════
void screenClock() {
  tft.fillRect(0, BODY_Y, TFT_W, BODY_H, C_BG);

  struct tm ti;
  if (!getLocalTime(&ti)) {
    printCenter("Sincronizando NTP...", 120, C_MUTED, 1);
    return;
  }

  char hmBuf[6];
  snprintf(hmBuf, 6, "%02d:%02d", ti.tm_hour, ti.tm_min);

  tft.setTextSize(6);
  int16_t bx, by; uint16_t bwHM, bhHM;
  tft.getTextBounds(hmBuf, 0, 0, &bx, &by, &bwHM, &bhHM);
  int startX = (TFT_W - (int)bwHM) / 2 - 5;
  int baseY  = 46;

  tft.setTextColor(C_WHITE, C_BG);
  tft.setCursor(startX, baseY);
  tft.print(hmBuf);

  // Fecha
  const char* dias[]  = {"Domingo","Lunes","Martes","Miercoles","Jueves","Viernes","Sabado"};
  const char* meses[] = {"Enero","Febrero","Marzo","Abril","Mayo","Junio",
                         "Julio","Agosto","Septiembre","Octubre","Noviembre","Diciembre"};
  char dBuf[40];
  snprintf(dBuf, 40, "%s %d de %s %d",
```

```
        dias[ti.tm_wday], ti.tm_mday, meses[ti.tm_mon], ti.tm_year+1900);
  printCenter(dBuf, 100, C_WHITE, 2);

  tft.drawFastHLine(20, 114, TFT_W-40, C_BORDER);

  // Tarjeta Exterior
  drawCard(8, 119, 148, 115, C_CARD);
  tft.setTextSize(1); tft.setTextColor(C_WHITE, C_CARD);
  tft.setCursor(16, 129); tft.print("EXTERIOR");

  char tBuf[8]; snprintf(tBuf, 8, "%.1f", W.tExt);
  tft.setTextSize(4);
  uint16_t tc = (W.tExt > 28) ? C_ORANGE : (W.tExt < 10) ? C_ACCENT : C_WHITE;
  tft.setTextColor(tc, C_CARD);
  tft.setCursor(16, 143); tft.print(tBuf);
  printDegC(tft.getCursorX(), 143, tc, C_CARD, 2);

  char descShort[16]; strlcpy(descShort, W.desc, 16);
  tft.setTextSize(1); tft.setTextColor(C_LGRAY, C_CARD);
  tft.setCursor(16, 200); tft.print(descShort);
  char hBf[10]; snprintf(hBf, 10, "Hum: %d%%", W.hum);
  tft.setCursor(16, 214); tft.print(hBf);
  drawIcon(125, 185, widType(W.wid), 12);

  // Tarjeta Interior
  drawCard(164, 119, 148, 115, C_CARD);
  tft.setTextSize(1); tft.setTextColor(C_WHITE, C_CARD);
  tft.setCursor(172, 129); tft.print("INTERIOR");

  if (W.dhtOk) {
    char tiBuf[8]; snprintf(tiBuf, 8, "%.1f", W.tInt);
    tft.setTextSize(4); tft.setTextColor(C_GREEN, C_CARD);
    tft.setCursor(172, 143); tft.print(tiBuf);
    printDegC(tft.getCursorX(), 143, C_GREEN, C_CARD, 2);
    tft.setTextSize(1); tft.setTextColor(C_LGRAY, C_CARD);
    char hiBuf[12]; snprintf(hiBuf, 12, "Hum: %.0f%%", W.hInt);
    tft.setCursor(172, 200); tft.print(hiBuf);
    tft.setTextColor(C_GREEN, C_CARD);
    tft.setCursor(172, 214); tft.print("DHT11 OK");
  } else {
    tft.setTextSize(1); tft.setTextColor(C_LGRAY, C_CARD);
    tft.setCursor(172, 163); tft.print("Sin datos");
    tft.setTextColor(C_RED, C_CARD);
    tft.setCursor(172, 178); tft.print("DHT11 error");
  }
}

//═══════════════════════════════════════════════════════════════════════════
//  PANTALLA 1 — INTERIOR
//═══════════════════════════════════════════════════════════════════════════
void screenInterior() {
  tft.fillRect(0, BODY_Y, TFT_W, BODY_H, C_BG);
  tft.drawFastHLine(0, 52, TFT_W, C_BORDER);
  printCenter("SENSOR INTERIOR  DHT11", 40, C_LGRAY, 1);
```

```
if (!W.dhtOk) {
  drawCard(60, 100, 200, 80, C_CARD);
  printCenterBg("DHT11 sin datos", 133, C_RED, C_CARD, 1);
  printCenterBg("Comprobar GPIO 27", 148, C_LGRAY, C_CARD, 1);
  return;
}

int16_t bx, by; uint16_t bw, bh;

drawCard(8, 56, 195, 178, C_CARD);
tft.setTextSize(1); tft.setTextColor(C_WHITE, C_CARD);
tft.setCursor(16, 66); tft.print("TEMPERATURA");

char tBuf[8]; snprintf(tBuf, 8, "%.1f", W.tInt);
tft.setTextSize(6); tft.setTextColor(C_GREEN, C_CARD);
tft.getTextBounds(tBuf, 0, 0, &bx, &by, &bw, &bh);
tft.setCursor(16 + (163 - (int)bw) / 2, 84);
tft.print(tBuf);
printDegC(tft.getCursorX(), 84, C_GREEN, C_CARD, 2);

// Barra de confort
int bX=16, bY=188, bW=163, bH=12;
tft.drawRoundRect(bX, bY, bW, bH, 4, C_BORDER);
float pct = constrain((W.tInt - 10.f) / 20.f, 0.f, 1.f);
uint16_t bc = (W.tInt < 16) ? C_BLUE : (W.tInt > 26) ? C_ORANGE : C_GREEN;
tft.fillRoundRect(bX+1, bY+1, (int)(pct*(bW-2)), bH-2, 3, bc);
tft.setTextSize(1); tft.setTextColor(C_LGRAY, C_CARD);
tft.setCursor(bX, bY+16);      tft.print("Frio");
tft.setCursor(bX+bW-24, bY+16); tft.print("Calor");
printCenterBg("CONFORT", bY+16, C_WHITE, C_CARD, 1, 8, 195);

// Offset activo — muestra el valor configurado
char offBuf[18];
snprintf(offBuf, 18, "Offset: %+.1f", cfg_tOffset);
tft.setTextSize(1); tft.setTextColor(C_MUTED, C_CARD);
tft.setCursor(16, bY+28); tft.print(offBuf);

drawCard(209, 56, 103, 88, C_CARD2);
tft.setTextSize(1); tft.setTextColor(C_WHITE, C_CARD2);
tft.setCursor(217, 66); tft.print("HUMEDAD");
char hBuf[8]; snprintf(hBuf, 8, "%.0f%%", W.hInt);
tft.setTextSize(3); tft.setTextColor(C_ACCENT, C_CARD2);
tft.getTextBounds(hBuf, 0, 0, &bx, &by, &bw, &bh);
tft.setCursor(209 + (103-(int)bw)/2, 88);
tft.print(hBuf);
tft.setTextSize(1); tft.setTextColor(C_LGRAY, C_CARD2);
tft.setCursor(217, 132); tft.print("Hum. relativa");

drawCard(209, 150, 103, 84, C_CARD2);
tft.setTextSize(1); tft.setTextColor(C_WHITE, C_CARD2);
tft.setCursor(217, 160); tft.print("SENSOR");
tft.setTextColor(C_GREEN, C_CARD2);
tft.setCursor(217, 176); tft.print("DHT11");
```

```
  tft.setTextColor(C_LGRAY, C_CARD2);
  tft.setCursor(217, 192); tft.print("GPIO 27");
  tft.setTextColor(C_GREEN, C_CARD2);
  tft.setCursor(217, 208); tft.print("Estado: OK");
}

//═══════════════════════════════════════════════════════════════════════════
//  PANTALLA 2 — EXTERIOR
//═══════════════════════════════════════════════════════════════════════════
void screenExterior() {
  tft.fillRect(0, BODY_Y, TFT_W, BODY_H, C_BG);
  tft.drawFastHLine(0, 52, TFT_W, C_BORDER);
  printCenter("EXTERIOR  OpenWeatherMap", 40, C_LGRAY, 1);

  int16_t bx, by; uint16_t bw, bh;

  drawCard(8, 56, 132, 178, C_CARD);
  char wt = widType(W.wid);
  drawIcon(74, 118, wt, 30);
  char descShort[16]; strlcpy(descShort, W.desc, 16);
  printCenterBg(descShort, 168, C_WHITE, C_CARD, 1, 8, 132);

  tft.setTextSize(2); tft.setTextColor(C_LGRAY, C_CARD);
  tft.setCursor(20, 196); tft.print("HOY ");
  tft.setTextColor(C_ORANGE, C_CARD);
  char mxBuf[6]; snprintf(mxBuf, 6, "%d", W.todayMax);
  tft.print(mxBuf);
  printDeg(tft.getCursorX(), 193, C_ORANGE, C_CARD, 1);
  tft.setCursor(tft.getCursorX() + 6, 196);
  tft.setTextColor(C_LGRAY, C_CARD); tft.print("/");
  tft.setTextColor(C_ACCENT, C_CARD);
  char mnBuf[6]; snprintf(mnBuf, 6, "%d", W.todayMin);
  tft.print(mnBuf);
  printDeg(tft.getCursorX(), 193, C_ACCENT, C_CARD, 1);

  drawCard(146, 56, 166, 88, C_CARD);
  tft.setTextSize(1); tft.setTextColor(C_WHITE, C_CARD);
  tft.setCursor(154, 66); tft.print("TEMPERATURA EXTERIOR");

  char tBuf[8]; snprintf(tBuf, 8, "%.1f", W.tExt);
  tft.setTextSize(5);
  uint16_t tc = (W.tExt > 28) ? C_ORANGE : (W.tExt < 10) ? C_ACCENT : C_WHITE;
  tft.setTextColor(tc, C_CARD);
  tft.getTextBounds(tBuf, 0, 0, &bx, &by, &bw, &bh);
  tft.setCursor(146 + (166 - (int)bw)/2 - 16, 80);
  tft.print(tBuf);
  printDegC(tft.getCursorX(), 80, tc, C_CARD, 2);

  drawCard(146, 150, 80, 84, C_CARD2);
  tft.setTextSize(1); tft.setTextColor(C_WHITE, C_CARD2);
  tft.setCursor(152, 160); tft.print("HUMEDAD");
  char hBuf[8]; snprintf(hBuf, 8, "%d%%", W.hum);
  tft.setTextSize(3); tft.setTextColor(C_ACCENT, C_CARD2);
  tft.getTextBounds(hBuf, 0, 0, &bx, &by, &bw, &bh);
```

```
  tft.setCursor(146 + (80-(int)bw)/2, 178);
  tft.print(hBuf);
  tft.setTextSize(1); tft.setTextColor(C_LGRAY, C_CARD2);
  tft.setCursor(152, 220); tft.print("% HR ext.");

  drawCard(232, 150, 80, 84, C_CARD2);
  tft.setTextSize(1); tft.setTextColor(C_WHITE, C_CARD2);
  tft.setCursor(238, 160); tft.print("PRESION");
  char pBuf[8]; snprintf(pBuf, 8, "%.0f", W.pressure);
  tft.setTextSize(2); tft.setTextColor(C_YELLOW, C_CARD2);
  tft.getTextBounds(pBuf, 0, 0, &bx, &by, &bw, &bh);
  tft.setCursor(232 + (80-(int)bw)/2, 184);
  tft.print(pBuf);
  tft.setTextSize(1); tft.setTextColor(C_LGRAY, C_CARD2);
  tft.setCursor(238, 218); tft.print("hPa");
}



//═══════════════════════════════════════════════════════════════════════════
//  PANTALLA 3 — DETALLES
//═══════════════════════════════════════════════════════════════════════════
void screenDetails() {
  tft.fillRect(0, BODY_Y, TFT_W, BODY_H, C_BG);
  tft.drawFastHLine(0, 52, TFT_W, C_BORDER);
  printCenter("DETALLES METEOROLOGICOS", 40, C_LGRAY, 1);

  int16_t bx, by; uint16_t bw, bh;

  drawCard(8, 56, 150, 86, C_CARD);
  tft.setTextSize(1); tft.setTextColor(C_WHITE, C_CARD);
  tft.setCursor(16, 66); tft.print("PRESION ATMOSFERICA");
  char pBuf[8]; snprintf(pBuf, 8, "%.0f", W.pressure);
  tft.setTextSize(4); tft.setTextColor(C_YELLOW, C_CARD);
  tft.setCursor(16, 82); tft.print(pBuf);
  tft.setTextSize(1); tft.setTextColor(C_LGRAY, C_CARD);
  tft.setCursor(16, 133); tft.print("hPa");

  drawCard(164, 56, 148, 86, C_CARD);
  tft.setTextSize(1); tft.setTextColor(C_WHITE, C_CARD);
  tft.setCursor(172, 66); tft.print("HUMEDAD EXTERIOR");
  char hBuf[8]; snprintf(hBuf, 8, "%d%%", W.hum);
  tft.setTextSize(4); tft.setTextColor(C_ACCENT, C_CARD);
  tft.setCursor(172, 82); tft.print(hBuf);
  tft.setTextSize(1); tft.setTextColor(C_LGRAY, C_CARD);
  tft.setCursor(172, 133); tft.print("% HR");

  drawCard(8, 148, 304, 62, C_CARD);
  tft.setTextSize(1); tft.setTextColor(C_WHITE, C_CARD);
  tft.setCursor(16, 158); tft.print("VELOCIDAD VIENTO");
  char wBuf[18]; snprintf(wBuf, 18, "%d km/h", W.wind);
  tft.setTextSize(2); tft.setTextColor(C_YELLOW, C_CARD);
  tft.setCursor(16, 172); tft.print(wBuf);
  tft.setTextSize(1); tft.setTextColor(C_WHITE, C_CARD);
```

```
  tft.setCursor(170, 158); tft.print("DIRECCION");
  char dBuf[16]; snprintf(dBuf, 16, "%s  (%d)", degToDir(W.windDeg), W.windDeg);
  tft.setTextSize(2); tft.setTextColor(C_YELLOW, C_CARD);
  tft.setCursor(170, 172); tft.print(dBuf);

  drawCard(8, 216, 148, 18, C_PANEL);
  tft.setTextSize(1); tft.setTextColor(C_LGRAY, C_PANEL);
  tft.setCursor(14, 221);
  tft.print("IP: "); tft.print(WiFi.localIP());

  drawCard(162, 216, 150, 18, C_PANEL);
  unsigned long up = millis() / 1000;
  char upBuf[24];
  snprintf(upBuf, 24, "Up: %luh %02lum %02lus", up/3600, (up%3600)/60, up%60);
  tft.setTextSize(1); tft.setTextColor(C_LGRAY, C_PANEL);
  tft.setCursor(168, 221); tft.print(upBuf);
}

// ═══════════════════════════════════════════════════════════════════════
//  PANTALLA 4 — PREVISION 3 DIAS
// ═══════════════════════════════════════════════════════════════════════
void screenForecast() {
  tft.fillRect(0, BODY_Y, TFT_W, BODY_H, C_BG);
  tft.drawFastHLine(0, 52, TFT_W, C_BORDER);
  printCenter("PREVISION 3 DIAS", 40, C_LGRAY, 1);

  int16_t bx, by; uint16_t bw, bh;
  int colW = 96, gap = 8, startX = 8;

  for (int i = 0; i < 3; i++) {
    int cx = startX + i*(colW + gap);
    drawCard(cx, 56, colW, 178, C_CARD);
    // Dia de la semana (destacado) + fecha DD/MM (atenuada)
    printCenterBg(W.fc[i].diaSem, 64, C_WHITE,  C_CARD, 1, cx, colW);
    printCenterBg(W.fc[i].fecha,  74, C_WHITE,  C_CARD, 1, cx, colW);

    char wt = widType(W.fc[i].wid);
    drawIcon(cx + colW/2, 124, wt, 26);

    int degW = 2*2 + 1 + 5 + 12;

    char mxBuf[6]; snprintf(mxBuf, 6, "%d", W.fc[i].tMax);
    tft.setTextSize(2);
    tft.getTextBounds(mxBuf, 0, 0, &bx, &by, &bw, &bh);
    tft.setTextColor(C_ORANGE, C_CARD);
    tft.setCursor(cx + (colW - (int)bw - degW)/2, 163);
    tft.print(mxBuf);
    printDegC(tft.getCursorX(), 163, C_ORANGE, C_CARD, 2);

    tft.drawFastHLine(cx+8, 183, colW-16, C_BORDER);

    char mnBuf[6]; snprintf(mnBuf, 6, "%d", W.fc[i].tMin);
    tft.setTextSize(2);
    tft.getTextBounds(mnBuf, 0, 0, &bx, &by, &bw, &bh);
```

```
    tft.setTextColor(C_ACCENT, C_CARD);
    tft.setCursor(cx + (colW - (int)bw - degW)/2, 187);
    tft.print(mnBuf);
    printDegC(tft.getCursorX(), 187, C_ACCENT, C_CARD, 2);

    const char* wDesc = "--";
    switch(wt) {
      case 's': wDesc="Despejado"; break;
      case 'p': wDesc="P.nubes";   break;
      case 'm': wDesc="Dispersas"; break;
      case 'c': wDesc="Nublado";   break;
      case 'd': wDesc="Llovizna";  break;
      case 'r': wDesc="Lluvia";    break;
      case 'n': wDesc="Nieve";     break;
      case 'f': wDesc="Niebla";    break;
      case 't': wDesc="Tormenta";  break;
    }
    printCenterBg(wDesc, 212, C_LGRAY, C_CARD, 1, cx, colW);
  }
}

//═══════════════════════════════════════════════════════════════════════════
//  OWM — Tiempo actual
//  NOTA: no tocar cfg_tOffset aqui, solo datos meteorologicos
//═══════════════════════════════════════════════════════════════════════════
bool fetchOWM() {
  if (WiFi.status() != WL_CONNECTED) return false;
  HTTPClient http;
  String url = "http://api.openweathermap.org/data/2.5/weather?q="
             + String(cfg_city) + "," + String(cfg_country)
             + "&appid=" + String(OWM_KEY) + "&units=metric&lang=es";
  http.begin(url); http.setTimeout(8000);
  if (http.GET() == 200) {
    DynamicJsonDocument doc(2048);
    deserializeJson(doc, http.getString());
    strlcpy(W.city,  doc["name"] | cfg_city, 20);
    W.tExt    = doc["main"]["temp"];
    W.hum     = doc["main"]["humidity"];
    W.pressure= doc["main"]["pressure"];
    W.wind    = (int)((float)doc["wind"]["speed"] * 3.6f);
    W.windDeg = doc["wind"]["deg"] | 0;
    W.wid     = doc["weather"][0]["id"] | 800;
    strlcpy(W.desc, doc["weather"][0]["description"] | "", 32);
    if (W.desc[0]) W.desc[0] = toupper(W.desc[0]);
    W.owmOk = true;
  }
  http.end();
  return W.owmOk;
}

//═══════════════════════════════════════════════════════════════════════════
//  OWM — Prevision 3 dias
//═══════════════════════════════════════════════════════════════════════════
bool fetchForecast() {
```

```
  if (WiFi.status() != WL_CONNECTED) return false;
  HTTPClient http;
  String url = "http://api.openweathermap.org/data/2.5/forecast?q="
            + String(cfg_city) + "," + String(cfg_country)
            + "&appid=" + String(OWM_KEY) + "&units=metric&cnt=32";
  http.begin(url); http.setTimeout(10000);
  if (http.GET() != 200) { http.end(); return false; }
  DynamicJsonDocument doc(16384);
  if (deserializeJson(doc, http.getStream())) { http.end(); return false; }
  http.end();

  struct tm now; getLocalTime(&now, 1000);
  int today = now.tm_mday;
  float todMax=-99, todMin=99; int todWid=800;
  float dMax[3]={-99,-99,-99}, dMin[3]={99,99,99};
  bool  dSet[3]={false,false,false};
  int   slot=-1, lastDay=-1;

  for (JsonObject item : doc["list"].as<JsonArray>()) {
    time_t ts = (time_t)(long)item["dt"];
    struct tm* lt = localtime(&ts);
    int d = lt->tm_mday;
    float t = item["main"]["temp"];
    int wid = item["weather"][0]["id"] | 800;
    if (d == today) {
      if (t > todMax) todMax=t;
      if (t < todMin) todMin=t;
      todWid = wid;
      continue;
    }
    if (d != lastDay) {
      if (slot < 2) { slot++; lastDay=d; } else break;
    }
    if (!dSet[slot]) {
      const char* diasSem[] = {"Dom","Lun","Mar","Mie","Jue","Vie","Sab"};
      snprintf(W.fc[slot].fecha,  6, "%02d/%02d", lt->tm_mday, lt->tm_mon+1);
      strlcpy(W.fc[slot].diaSem, diasSem[lt->tm_wday], 5);
      dSet[slot] = true;
    }
    if (t > dMax[slot]) dMax[slot]=t;
    if (t < dMin[slot]) dMin[slot]=t;
    W.fc[slot].wid = wid;
  }
  W.todayMax = (int)roundf(todMax);
  W.todayMin = (int)roundf(todMin);
  W.todayWid = todWid;
  for (int i=0; i<3; i++) {
    W.fc[i].tMax = (int)roundf(dMax[i]);
    W.fc[i].tMin = (int)roundf(dMin[i]);
  }
  return true;
}

//═══════════════════════════════════════════════════════════════════════════
```

```
//  API REST
//═══════════════════════════════════════════════════════════════════════════
void setCORS() {
  server.sendHeader("Access-Control-Allow-Origin", "*");
  server.sendHeader("Access-Control-Allow-Methods", "GET,POST,OPTIONS");
  server.sendHeader("Access-Control-Allow-Headers", "Content-Type");
}

void handleData() {
  setCORS();
  struct tm ti; bool timeOk = getLocalTime(&ti, 500);
  char timeBuf[20]="--:--:--", dateBuf[12]="--/--/----";
  if (timeOk) {
    snprintf(timeBuf, 20, "%02d:%02d:%02d", ti.tm_hour, ti.tm_min, ti.tm_sec);
    snprintf(dateBuf, 12, "%02d/%02d/%04d", ti.tm_mday, ti.tm_mon+1, ti.tm_year+1900);
  }
  DynamicJsonDocument doc(1024);
  doc["time"]     = timeBuf;
  doc["date"]     = dateBuf;
  doc["city"]     = W.city;
  doc["tExt"]     = serialized(String(W.tExt, 1));
  doc["tInt"]     = serialized(String(W.tInt, 1));
  doc["hExt"]     = W.hum;
  doc["hInt"]     = W.dhtOk ? (int)roundf(W.hInt) : -1;
  doc["pressure"] = serialized(String(W.pressure, 0));
  doc["wind"]     = W.wind;
  doc["windDir"]  = degToDir(W.windDeg);
  doc["windDeg"]  = W.windDeg;
  doc["desc"]     = W.desc;
  doc["wid"]      = W.wid;
  doc["todayMax"] = W.todayMax;
  doc["todayMin"] = W.todayMin;
  doc["wifiOk"]   = W.wifiOk;
  doc["owmOk"]    = W.owmOk;
  doc["dhtOk"]    = W.dhtOk;
  doc["tOffset"]    = cfg_tOffset;
  doc["winterTime"] = cfg_winterTime;
  doc["ip"]        = WiFi.localIP().toString();
  doc["uptime"]   = millis() / 1000;
  JsonArray fc = doc.createNestedArray("forecast");
  for (int i=0; i<3; i++) {
    JsonObject d = fc.createNestedObject();
    d["date"]=W.fc[i].fecha; d["max"]=W.fc[i].tMax;
    d["min"]=W.fc[i].tMin;   d["wid"]=W.fc[i].wid;
  }
  String out; serializeJson(doc, out);
  server.send(200, "application/json", out);
}

void handleHistory() {
  setCORS();
  String out = "[";
  int start = (histCount < HIST_SIZE) ? 0 : histHead;
  for (int i=0; i<histCount; i++) {
```

```
    int idx = (start+i) % HIST_SIZE;
    if (i>0) out += ",";
    out += "{\"ts\":"+String(hist[idx].ts)
        + ",\"tExt\":"+String(hist[idx].tExt,1)
        + ",\"tInt\":"+String(hist[idx].tInt,1)
        + ",\"hum\":"+String(hist[idx].hum)
        + ",\"pres\":"+String(hist[idx].pressure,0)+"}";
  }
  out += "]";
  server.send(200, "application/json", out);
}

void handleLogin() {
  setCORS();
  if (!server.hasArg("plain")) { server.send(400,"application/json","{\"error\":\"no body\"}");
return; }
  DynamicJsonDocument doc(256);
  if (deserializeJson(doc, server.arg("plain"))) {
server.send(400,"application/json","{\"error\":\"json\"}"); return; }
  const char* u = doc["user"] | "";
  const char* p = doc["pass"] | "";
  if (strcmp(u, CFG_USER)==0 && strcmp(p, CFG_PASS)==0)
    server.send(200, "application/json", "{\"ok\":true,\"token\":\"est2026ok\"}");
  else
    server.send(401, "application/json", "{\"ok\":false,\"error\":\"Credenciales incorrectas\"}");
}

void handleConfig() {
  setCORS();
  if (!server.hasArg("plain")) { server.send(400,"application/json","{\"error\":\"no body\"}");
return; }
  DynamicJsonDocument doc(512);
  if (deserializeJson(doc, server.arg("plain"))) {
server.send(400,"application/json","{\"error\":\"json\"}"); return; }
  const char* tok = doc["token"] | "";
  if (strcmp(tok, "est2026ok") != 0) { server.send(401,"application/json","{\"error\":\"No
autorizado\"}"); return; }
  bool changed = false;
  if (doc.containsKey("city"))    { strlcpy(cfg_city,    doc["city"],    32); changed=true; }
  if (doc.containsKey("country")) { strlcpy(cfg_country, doc["country"], 8);  changed=true; }
  if (doc.containsKey("ssid"))    { strlcpy(cfg_ssid,    doc["ssid"],    32); changed=true; }
  if (doc.containsKey("pass"))    { strlcpy(cfg_pass,    doc["pass"],    64); changed=true; }
  if (doc.containsKey("tOffset"))    { cfg_tOffset    = doc["tOffset"].as<float>();  changed=true; }
  if (doc.containsKey("winterTime")) { cfg_winterTime = doc["winterTime"].as<bool>();
changed=true; }
  if (changed) saveConfig();
  server.send(200, "application/json", "{\"ok\":true}");
}

void handleReboot() {
  setCORS();
  server.send(200, "application/json", "{\"ok\":true}");
  delay(500); ESP.restart();
}
```

```
void handleOptions() { setCORS(); server.send(204); }

//═══════════════════════════════════════════════════════════════════════════
//  PWA HTML
//═══════════════════════════════════════════════════════════════════════════
const char PWA_HTML[] PROGMEM = R"rawhtml(
<!DOCTYPE html>
<html lang="es">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width,initial-scale=1,maximum-scale=1">
<meta name="theme-color" content="#0a1628">
<meta name="apple-mobile-web-app-capable" content="yes">
<title>Estacion Met.</title>
<style>
:root{--sky:#0a1628;--panel:#0d2040;--card:#102550;--accent:#00ccff;--green:#00e676;--
yellow:#ffd600;--orange:#ff9100;--red:#ff1744;--text:#e0f0ff;--muted:#5580aa}
*{box-sizing:border-box;margin:0;padding:0}
body{background:var(--sky);color:var(--text);font-family:'Segoe UI',sans-serif;min-
height:100vh;padding-bottom:80px}
header{background:var(--panel);padding:12px 16px;display:flex;align-items:center;justify-
content:space-between;border-bottom:1px solid #1a3a60;position:sticky;top:0;z-index:10}
header h1{font-size:16px;font-weight:600;color:var(--accent)}
.status-dots{display:flex;gap:8px;align-items:center}
.dot{width:8px;height:8px;border-radius:50%}
.dot.ok{background:var(--green)}.dot.err{background:var(--red)}.dot.off{background:#334}
.dot-label{font-size:10px;color:var(--muted)}
nav{display:flex;background:var(--panel);border-bottom:1px solid #1a3a60}
nav button{flex:1;padding:10px 4px;background:none;border:none;color:var(--muted);font-
size:12px;cursor:pointer;border-bottom:2px solid transparent;transition:.2s}
nav button.active{color:var(--accent);border-bottom-color:var(--accent)}
.page{display:none;padding:12px}.page.active{display:block}
.grid2{display:grid;grid-template-columns:1fr 1fr;gap:10px;margin-bottom:10px}
.grid3{display:grid;grid-template-columns:1fr 1fr 1fr;gap:8px;margin-bottom:10px}
.card{background:var(--card);border-radius:12px;padding:14px;border:1px solid #1a3a60}
.card-label{font-size:10px;color:var(--muted);text-transform:uppercase;letter-
spacing:.5px;margin-bottom:4px}
.card-val{font-size:28px;font-weight:700;color:var(--accent);line-height:1}
.card-val.warm{color:var(--orange)}.card-val.cool{color:#44aaff}
.card-unit{font-size:13px;color:var(--muted);margin-top:2px}
.hero{background:var(--card);border-radius:14px;padding:18px;margin-
bottom:10px;border:1px solid #1a3a60;display:flex;justify-content:space-between;align-
items:center}
.hero-temp{font-size:64px;font-weight:800;line-height:1}
.hero-right{text-align:right}
.hero-city{font-size:18px;font-weight:600;color:var(--accent)}
.hero-time{font-size:32px;font-weight:700;color:var(--text);margin-top:8px}
.hero-date{font-size:12px;color:var(--muted)}
.fc-card{background:var(--card);border-radius:12px;padding:10px;text-align:center;border:1px
solid #1a3a60}
.fc-date{font-size:11px;color:var(--muted);margin-bottom:4px}
.fc-icon{font-size:24px;margin:4px 0}
.fc-temps{font-size:13px;font-weight:600}
.fc-max{color:var(--orange)}.fc-min{color:var(--accent)}
```

```
.chart-wrap{background:var(--card);border-radius:12px;padding:14px;margin-
bottom:10px;border:1px solid #1a3a60}
.chart-title{font-size:12px;color:var(--muted);margin-bottom:8px}
canvas{width:100%!important;height:160px!important}
.form-group{margin-bottom:12px}
.form-group label{display:block;font-size:12px;color:var(--muted);margin-bottom:4px}
.form-group input{width:100%;background:#0a1e38;border:1px solid #1a3a60;border-
radius:8px;padding:10px 12px;color:var(--text);font-size:14px;outline:none}
.form-group input:focus{border-color:var(--accent)}
.form-group input.short{width:120px}
.offset-hint{font-size:11px;color:var(--muted);margin-left:8px}
.btn{width:100%;padding:12px;border-radius:10px;border:none;font-size:15px;font-
weight:600;cursor:pointer;margin-top:6px}
.btn-primary{background:var(--accent);color:#000}
.btn-danger{background:var(--red);color:#fff;margin-top:10px}
.btn-logout{background:#1a3a60;color:var(--muted);margin-top:6px}
.btn:active{opacity:.8}
.msg{padding:10px;border-radius:8px;font-size:13px;margin-top:8px;text-align:center}
.msg.ok{background:#0d3020;color:var(--green);border:1px solid var(--green)}
.msg.err{background:#300a0a;color:var(--red);border:1px solid var(--red)}
.int-badge{background:#0a2a1a;border:1px solid #1a5a3a;border-
radius:12px;padding:14px;margin-bottom:10px;display:flex;gap:16px;align-items:center}
.int-ico{font-size:28px}
.int-data{flex:1}
.int-row{font-size:13px;color:var(--muted)}
.int-val{font-size:20px;font-weight:700;color:var(--green)}
.info-row{display:flex;justify-content:space-between;padding:8px 0;border-bottom:1px solid
#1a3a60;font-size:13px}
.info-row:last-child{border:none}
.info-key{color:var(--muted)}
.login-wrap{display:flex;flex-direction:column;align-items:center;justify-
content:center;padding:40px 20px}
.login-icon{font-size:48px;margin-bottom:16px}
.login-title{font-size:18px;font-weight:700;color:var(--accent);margin-bottom:6px}
.login-sub{font-size:12px;color:var(--muted);margin-bottom:24px}
.login-box{width:100%;max-width:320px}
.footer{text-align:center;padding:18px 12px 8px;border-top:1px solid #1a3a60;margin-
top:10px}
.footer-title{font-size:22px;font-weight:800;color:var(--accent);letter-spacing:1px;margin-
bottom:10px}
.footer-line{font-size:11px;color:var(--muted);margin:3px 0}
.footer-line span{color:var(--text)}
.toggle-row{display:flex;align-items:center;justify-content:space-between;padding:10px
0;border-bottom:1px solid #1a3a60}
.toggle-row:last-of-type{border-bottom:none}
.toggle-label{font-size:13px;color:var(--text)}
.toggle-sub{font-size:11px;color:var(--muted);margin-top:2px}
.toggle-switch{position:relative;width:44px;height:24px;flex-shrink:0}
.toggle-switch input{opacity:0;width:0;height:0}
.toggle-slider{position:absolute;inset:0;background:#1a3a60;border-
radius:24px;cursor:pointer;transition:.3s}
.toggle-
slider:before{content:"";position:absolute;width:18px;height:18px;left:3px;top:3px;backgroun
d:#fff;border-radius:50%;transition:.3s}
```

```
.toggle-switch input:checked+.toggle-slider{background:var(--accent)}
.toggle-switch input:checked+.toggle-slider:before{transform:translateX(20px)}
</style>
</head>
<body>
<header>
  <h1 id="hCity">Estacion Met.</h1>
  <div class="status-dots">
    <span class="dot-label">WiFi</span><div class="dot off" id="dotWifi"></div>
    <span class="dot-label">OWM</span><div class="dot off" id="dotOwm"></div>
    <span class="dot-label">DHT</span><div class="dot off" id="dotDht"></div>
  </div>
</header>
<nav>
  <button class="active" onclick="showPage('dash',this)">&#9730; Inicio</button>
  <button onclick="showPage('detail',this)">&#9729; Detalle</button>
  <button onclick="showPage('history',this)">&#128202; Historial</button>
  <button onclick="showPage('config',this)">&#9881; Config</button>
</nav>

<!-- INICIO -->
<div class="page active" id="page-dash">
  <div class="hero">
    <div>
      <div style="font-size:11px;color:var(--muted)">EXTERIOR</div>
      <div class="hero-temp" id="hTemp">--</div>
      <div style="font-size:13px;color:var(--muted)" id="hDesc">--</div>
      <div style="font-size:12px;color:var(--muted);margin-top:4px" id="hMaxMin">--</div>
    </div>
    <div class="hero-right">
      <div class="hero-city" id="hCityR">--</div>
      <div class="hero-time" id="hTime">--:--</div>
      <div class="hero-date" id="hDate">--</div>
    </div>
  </div>
  <div class="int-badge">
    <div class="int-ico">&#127968;</div>
    <div class="int-data">
      <div class="int-row">INTERIOR</div>
      <div style="display:flex;gap:20px;margin-top:4px">
        <div><div class="int-val" id="iTint">--</div><div class="int-row">Temp</div></div>
        <div><div class="int-val" id="iHint">--</div><div class="int-row">Hum</div></div>
      </div>
    </div>
  </div>
  <div class="grid3" id="fcRow">
    <div class="fc-card"><div class="fc-date">--/--</div><div class="fc-icon">&#9729;</div><div
class="fc-temps">--/--</div></div>
    <div class="fc-card"><div class="fc-date">--/--</div><div class="fc-icon">&#9729;</div><div
class="fc-temps">--/--</div></div>
    <div class="fc-card"><div class="fc-date">--/--</div><div class="fc-icon">&#9729;</div><div
class="fc-temps">--/--</div></div>
  </div>
  <div class="footer">
```

```
   <div class="footer-title">ESTACION METEOROLOGICA</div>
   <div class="footer-line">Firmware: <span>EstacionPWA v6 TFT ST7789V</span></div>
   <div class="footer-line">Creada por: <span>Ariel Izquierdo Ereza</span></div>
   <div class="footer-line"><span>2026</span></div>
  </div>
</div>

<!-- DETALLE -->
<div class="page" id="page-detail">
  <div class="grid2">
   <div class="card"><div class="card-label">Humedad ext</div><div class="card-val"
id="dHum">--</div><div class="card-unit">%</div></div>
   <div class="card"><div class="card-label">Presion</div><div class="card-val" id="dPres">--
</div><div class="card-unit">hPa</div></div>
  </div>
  <div class="grid2">
   <div class="card"><div class="card-label">Viento</div><div class="card-val" id="dWind">--
</div><div class="card-unit" id="dWindDir">--</div></div>
   <div class="card"><div class="card-label">Descripcion</div><div style="font-size:15px;font-
weight:600;margin-top:8px" id="dDesc">--</div></div>
  </div>
  <div class="card" style="margin-bottom:10px">
   <div class="card-label" style="margin-bottom:12px">Sistema</div>
   <div class="info-row"><span class="info-key">IP local</span><span id="sIp">--
</span></div>
   <div class="info-row"><span class="info-key">Ciudad OWM</span><span id="sCity">--
</span></div>
   <div class="info-row"><span class="info-key">Uptime</span><span id="sUptime">--
</span></div>
   <div class="info-row"><span class="info-key">DHT11</span><span id="sDht">--
</span></div>
   <div class="info-row"><span class="info-key">Offset temp.</span><span id="sOffset">--
</span></div>
   <div class="info-row"><span class="info-key">Pantalla</span><span>TFT ST7789V
320x240</span></div>
  </div>
</div>

<!-- HISTORIAL -->
<div class="page" id="page-history">
  <div class="chart-wrap"><div class="chart-title">Temperatura interior (C)</div><canvas
id="chartTInt"></canvas></div>
  <div class="chart-wrap"><div class="chart-title">Temperatura exterior (C)</div><canvas
id="chartTExt"></canvas></div>
  <div class="chart-wrap"><div class="chart-title">Humedad exterior (%)</div><canvas
id="chartHum"></canvas></div>
  <div class="chart-wrap"><div class="chart-title">Presion (hPa)</div><canvas
id="chartPres"></canvas></div>
</div>

<!-- CONFIG -->
<div class="page" id="page-config">
  <div id="loginPanel">
   <div class="login-wrap">
```

```
    <div class="login-icon">&#128274;</div>
    <div class="login-title">Acceso restringido</div>
    <div class="login-sub">Introduce las credenciales para acceder a la configuracion</div>
    <div class="login-box">
      <div class="form-group"><label>Usuario</label><input id="lgUser" type="text"
placeholder="Usuario" autocomplete="username"></div>
      <div class="form-group"><label>Contrasena</label><input id="lgPass" type="password"
placeholder="Contrasena" autocomplete="current-password"></div>
      <button class="btn btn-primary" onclick="doLogin()">&#128275; Entrar</button>
      <div id="lgMsg" style="display:none"></div>
    </div>
  </div>
 </div>
 <div id="cfgPanel" style="display:none">
  <div class="card">
    <div class="card-label" style="margin-bottom:12px">&#127759; Ubicacion OWM</div>
    <div class="form-group"><label>Ciudad</label><input id="cfgCity" type="text"
placeholder="Sabinanigo"></div>
    <div class="form-group"><label>Pais (codigo ISO)</label><input id="cfgCountry"
type="text" placeholder="ES" maxlength="3"></div>

    <div class="card-label" style="margin:12px 0">&#128246; Red WiFi</div>
    <div class="form-group"><label>SSID</label><input id="cfgSsid" type="text"
placeholder="Nombre red"></div>
    <div class="form-group"><label>Contrasena WiFi</label><input id="cfgWifiPass"
type="password" placeholder="Contrasena WiFi"></div>

    <div class="card-label" style="margin:12px 0">&#127777; Calibracion DHT11</div>
    <div class="form-group">
      <label>Offset temperatura interior (°C)</label>
      <div style="display:flex;align-items:center;gap:10px;margin-top:4px">
        <input id="cfgOffset" class="short" type="number" step="0.5" min="-10" max="10"
placeholder="0.0">
        <span class="offset-hint">Ej: -4 si el sensor marca 4° de mas</span>
      </div>
    </div>

    <div class="card-label" style="margin:12px 0">&#128336; Zona horaria</div>
    <div class="toggle-row">
      <div>
        <div class="toggle-label">Hora de invierno</div>
        <div class="toggle-sub">Activa: UTC+1 fijo  |  Inactiva: UTC+2
(verano)</div>
      </div>
      <label class="toggle-switch">
        <input type="checkbox" id="cfgWinterTime">
        <span class="toggle-slider"></span>
      </label>
    </div>

    <button class="btn btn-primary" onclick="saveConfig()">&#128190; Guardar y
reiniciar</button>
    <button class="btn btn-danger"  onclick="reboot()">&#9889; Reiniciar ESP32</button>
    <button class="btn btn-logout"  onclick="doLogout()">&#128274; Cerrar sesion</button>
```

```
    <div id="cfgMsg" style="display:none"></div>
   </div>
  </div>
</div>

<script>
let authToken = null;

function showPage(id, btn) {
  document.querySelectorAll('.page').forEach(p => p.classList.remove('active'));
  document.querySelectorAll('nav button').forEach(b => b.classList.remove('active'));
  document.getElementById('page-' + id).classList.add('active');
  btn.classList.add('active');
  if (id === 'history') loadHistory();
  if (id === 'config')  refreshConfigPanel();
}

function refreshConfigPanel() {
  document.getElementById('loginPanel').style.display = authToken ? 'none' : 'block';
  document.getElementById('cfgPanel').style.display   = authToken ? 'block' : 'none';
}

async function doLogin() {
  const user = document.getElementById('lgUser').value.trim();
  const pass = document.getElementById('lgPass').value;
  const msg  = document.getElementById('lgMsg');
  if (!user || !pass) { showMsg(msg, 'err', 'Introduce usuario y contrasena'); return; }
  try {
    const r = await fetch('/api/login', {
      method: 'POST',
      headers: {'Content-Type': 'application/json'},
      body: JSON.stringify({user, pass})
    });
    const d = await r.json();
    if (d.ok) {
      authToken = d.token;
      document.getElementById('lgUser').value = '';
      document.getElementById('lgPass').value = '';
      msg.style.display = 'none';
      refreshConfigPanel();
    } else {
      showMsg(msg, 'err', d.error || 'Credenciales incorrectas');
      authToken = null;
    }
  } catch(e) { showMsg(msg, 'err', 'Error de conexion'); }
}

document.addEventListener('DOMContentLoaded', () => {
  document.getElementById('lgPass').addEventListener('keydown', e => { if (e.key === 'Enter')
doLogin(); });
  document.getElementById('lgUser').addEventListener('keydown', e => { if (e.key === 'Enter')
document.getElementById('lgPass').focus(); });
});
```

```
function doLogout() {
  authToken = null;
  document.getElementById('cfgMsg').style.display = 'none';
  refreshConfigPanel();
}

function widIcon(wid) {
  if (wid >= 200 && wid < 300) return '&#9928;';
  if (wid >= 300 && wid < 600) return '&#127783;';
  if (wid >= 600 && wid < 700) return '&#10052;';
  if (wid === 800)             return '&#9728;';
  if (wid === 801 || wid === 802) return '&#9925;';
  return '&#9729;';
}

// — updateUI: actualiza TODA la interfaz con los datos de /api/data —
function updateUI(d) {
  document.getElementById('hCity').textContent  = d.city || 'Estacion Met.';
  document.getElementById('dotWifi').className  = 'dot ' + (d.wifiOk ? 'ok' : 'err');
  document.getElementById('dotOwm').className   = 'dot ' + (d.owmOk  ? 'ok' : 'err');
  document.getElementById('dotDht').className   = 'dot ' + (d.dhtOk  ? 'ok' : 'off');

  const t  = parseFloat(d.tExt);
  const el = document.getElementById('hTemp');
  el.textContent = t.toFixed(1) + '°C';
  el.className   = 'hero-temp ' + (t > 28 ? 'warm' : t < 10 ? 'cool' : '');

  document.getElementById('hDesc').textContent   = d.desc;
  document.getElementById('hMaxMin').textContent = '\u25b2' + d.todayMax + '\u00b0
\u25bc' + d.todayMin + '\u00b0';
  document.getElementById('hCityR').textContent  = d.city;
  document.getElementById('hTime').textContent   = d.time.substring(0, 5);
  document.getElementById('hDate').textContent   = d.date;

  document.getElementById('iTint').textContent = d.tInt !== undefined ?
parseFloat(d.tInt).toFixed(1) + '°C' : '--';
  document.getElementById('iHint').textContent = d.hInt >= 0 ? d.hInt + '%' : '--';

  const fc = d.forecast || [];
  document.getElementById('fcRow').innerHTML = fc.map(f => `
    <div class="fc-card">
      <div class="fc-date">${f.date}</div>
      <div class="fc-icon">${widIcon(f.wid)}</div>
      <div class="fc-temps"><span class="fc-max">${f.max}\u00b0</span>/<span class="fc-
min">${f.min}\u00b0</span></div>
    </div>`).join('');

  document.getElementById('dHum').textContent     = d.hExt + '%';
  document.getElementById('dPres').textContent    = d.pressure;
  document.getElementById('dWind').textContent    = d.wind;
  document.getElementById('dWindDir').textContent = d.windDir + ' (' + d.windDeg + '\u00b0)';
  document.getElementById('dDesc').textContent    = d.desc;
  document.getElementById('sIp').textContent      = d.ip;
  document.getElementById('sCity').textContent    = d.city;
```

```
  const up = d.uptime || 0;
  document.getElementById('sUptime').textContent  = `${Math.floor(up/3600)}h
${Math.floor((up%3600)/60)}m ${up%60}s`;
  document.getElementById('sDht').textContent     = d.dhtOk ? 'OK' : 'Sin datos';

  // Offset: mostrar en detalle y rellenar campo config si esta visible
  if (d.tOffset !== undefined) {
    document.getElementById('sOffset').textContent = (d.tOffset >= 0 ? '+' : '') +
parseFloat(d.tOffset).toFixed(1) + ' °C';
    const offEl = document.getElementById('cfgOffset');
    if (offEl) offEl.value = parseFloat(d.tOffset).toFixed(1);
  }
  // Hora invierno: rellenar checkbox
  const wtEl = document.getElementById('cfgWinterTime');
  if (wtEl && d.winterTime !== undefined) wtEl.checked = !!d.winterTime;
}

async function fetchData() {
  try {
    const r = await fetch('/api/data');
    if (!r.ok) return;
    updateUI(await r.json());
  } catch(e) { console.warn('fetchData', e); }
}

function drawChart(id, samples, key, color) {
  const cv = document.getElementById(id);
  if (!cv || !samples.length) return;
  const W = cv.width = cv.offsetWidth, H = cv.height = 160;
  const ctx = cv.getContext('2d');
  ctx.clearRect(0, 0, W, H);
  const vals = samples.map(s => parseFloat(s[key]));
  const mn = Math.min(...vals), mx = Math.max(...vals), rng = mx - mn || 1, pad = 16;
  ctx.strokeStyle = '#1a3a60'; ctx.lineWidth = 1;
  for (let i = 0; i <= 4; i++) {
    const y = pad + (H - pad*2)*i/4;
    ctx.beginPath(); ctx.moveTo(0, y); ctx.lineTo(W, y); ctx.stroke();
    ctx.fillStyle = '#5580aa'; ctx.font = '10px sans-serif'; ctx.textAlign = 'right';
    ctx.fillText((mx - rng*i/4).toFixed(1), W-2, y-2);
  }
  ctx.strokeStyle = color; ctx.lineWidth = 2; ctx.lineJoin = 'round'; ctx.beginPath();
  samples.forEach((s, i) => {
    const x = i*(W-1)/(samples.length-1);
    const y = pad + (H - pad*2)*(1 - (parseFloat(s[key]) - mn)/rng);
    i === 0 ? ctx.moveTo(x, y) : ctx.lineTo(x, y);
  });
  ctx.stroke();
  ctx.lineTo(W-1, H); ctx.lineTo(0, H); ctx.closePath();
  ctx.fillStyle = color.replace(')', ',0.15)').replace('rgb', 'rgba');
  ctx.fill();
}

async function loadHistory() {
  try {
```

```
    const r = await fetch('/api/history');
    if (!r.ok) return;
    const data = await r.json();
    if (!data.length) return;
    drawChart('chartTExt', data, 'tExt', 'rgb(255,145,0)');
    drawChart('chartTInt', data, 'tInt', 'rgb(0,230,118)');
    drawChart('chartHum',  data, 'hum',  'rgb(0,204,255)');
    drawChart('chartPres', data, 'pres', 'rgb(180,100,255)');
  } catch(e) { console.warn('loadHistory', e); }
}

function showMsg(el, type, txt) {
  el.style.display = 'block';
  el.className = 'msg ' + type;
  el.textContent = txt;
}

async function saveConfig() {
  if (!authToken) { showMsg(document.getElementById('cfgMsg'), 'err', 'Sesion expirada');
doLogout(); return; }

  // Leer offset — si el campo esta vacio o no es numero, usar 0
  const rawOffset = document.getElementById('cfgOffset').value;
  const tOffset   = rawOffset !== '' && !isNaN(parseFloat(rawOffset)) ? parseFloat(rawOffset) :
0;

  const body = {
    token:     authToken,
    city:      document.getElementById('cfgCity').value.trim(),
    country:   document.getElementById('cfgCountry').value.trim().toUpperCase(),
    ssid:      document.getElementById('cfgSsid').value,
    pass:      document.getElementById('cfgWifiPass').value,
    tOffset:   tOffset,
    winterTime: document.getElementById('cfgWinterTime').checked
  };

  // Eliminar campos vacios excepto token, tOffset y winterTime
  Object.keys(body).forEach(k => {
    if (k !== 'token' && k !== 'tOffset' && k !== 'winterTime' && !body[k]) delete body[k];
  });

  const msg = document.getElementById('cfgMsg');
  try {
    const r = await fetch('/api/config', {
      method: 'POST',
      headers: {'Content-Type': 'application/json'},
      body: JSON.stringify(body)
    });
    const d = await r.json();
    if (d.ok) {
      showMsg(msg, 'ok', 'Guardado. Reiniciando en 3s...');
      setTimeout(() => fetch('/api/reboot', {method: 'POST'}), 1500);
    } else {
      showMsg(msg, 'err', d.error || 'Error al guardar');
```

```
    }
  } catch(e) { showMsg(msg, 'err', 'Error de conexion'); }
}

async function reboot() {
  if (!confirm('Reiniciar el ESP32?')) return;
  await fetch('/api/reboot', {method: 'POST'});
}

fetchData();
setInterval(fetchData, 30000);
</script>
</body>
</html>
)rawhtml";

void handleRoot() {
  server.sendHeader("Cache-Control", "no-cache");
  server.send_P(200, "text/html", PWA_HTML);
}

//═══════════════════════════════════════════════════════════════════════════
//  PANTALLA DE ARRANQUE
//═══════════════════════════════════════════════════════════════════════════
void splashScreen() {
  tft.fillScreen(C_BG);
  printCenter("ESTACION", 20, C_ACCENT, 3);
  printCenter("METEOROLOGICA", 50, C_ACCENT, 3);
  tft.drawFastHLine(30, 84, TFT_W-60, C_BORDER);
  printCenter("ESP32  +  ST7789V  320x240", 94,  C_LGRAY, 1);
  printCenter("DHT11  +  OpenWeatherMap",   108, C_LGRAY, 1);
  printCenter("WebServer Puerto 80 - PWA",  122, C_LGRAY, 1);
  tft.drawFastHLine(30, 140, TFT_W-60, C_BORDER);
  printCenter("Ariel Izquierdo Ereza", 150, C_WHITE, 1);
  printCenter("2026", 165, C_LGRAY, 1);
  printCenter("Version: TFTb_V6", 180, C_LGRAY, 1);
}

void splashStatus(bool wifiOk) {
  tft.fillRect(0, 190, TFT_W, 50, C_BG);
  if (wifiOk) {
    printCenter("WiFi conectado", 196, C_GREEN, 1);
    String ip = "IP: " + WiFi.localIP().toString() + "  Puerto 80";
    printCenter(ip.c_str(), 210, C_LGRAY, 1);
    printCenter("Iniciando servicios...", 224, C_LGRAY, 1);
  } else {
    printCenter("Sin WiFi  --  Modo offline", 210, C_RED, 1);
  }
  delay(3000);
}
```

```
//═══════════════════════════════════════════════════════════════════════════
//  SETUP
//═══════════════════════════════════════════════════════════════════════════
void setup() {
  Serial.begin(115200);
  pinMode(LED_STATUS, OUTPUT);
  digitalWrite(LED_STATUS, LOW);
  pinMode(BTN_PIN, INPUT_PULLUP);

  loadConfig();

  tft.init(240, 320);
  tft.setRotation(3);
  tft.fillScreen(C_BG);
  tft.setTextWrap(false);
  tft.cp437(true);

  dht.begin();
  splashScreen();

  strlcpy(W.city, cfg_city, 20);
  for (int i=0; i<3; i++) {
    strlcpy(W.fc[i].fecha,  "--/--", 6);
    strlcpy(W.fc[i].diaSem, "---",   5);
    W.fc[i].tMax=0; W.fc[i].tMin=0; W.fc[i].wid=800;
  }

  WiFi.mode(WIFI_STA);
  WiFi.disconnect();
  delay(300);
  WiFi.begin(cfg_ssid, cfg_pass);
  int t = 0;
  while (WiFi.status() != WL_CONNECTED && t++ < 40) {
    digitalWrite(LED_STATUS, !digitalRead(LED_STATUS));
    delay(500);
    Serial.print(".");
  }
  W.wifiOk = (WiFi.status() == WL_CONNECTED);
  splashStatus(W.wifiOk);

  if (W.wifiOk) {
    digitalWrite(LED_STATUS, HIGH);
    // Hora de invierno: CET UTC+1 sin cambio horario de verano
    // Hora de verano:   CET-1CEST (automatico DST)
    const char* tzStr = cfg_winterTime
                      ? "CET-1"
                      : "CET-1CEST,M3.5.0,M10.5.0/3";
    configTzTime(tzStr, "pool.ntp.org", "time.google.com");
    struct tm ti; int nt=0;
    while (!getLocalTime(&ti, 2000) && nt++ < 3) delay(500);
    fetchOWM();
    fetchForecast();

    server.on("/",          HTTP_GET,     handleRoot);
```

```
    server.on("/api/data",    HTTP_GET,     handleData);
    server.on("/api/history", HTTP_GET,     handleHistory);
    server.on("/api/login",   HTTP_POST,    handleLogin);
    server.on("/api/config",  HTTP_POST,    handleConfig);
    server.on("/api/reboot",  HTTP_POST,    handleReboot);
    server.on("/api/config",  HTTP_OPTIONS, handleOptions);
    server.on("/api/reboot",  HTTP_OPTIONS, handleOptions);
    server.on("/api/login",   HTTP_OPTIONS, handleOptions);
    server.onNotFound([]() { server.send(404, "text/plain", "Not found"); });
    server.begin();
    Serial.println("\nWebServer: http://" + WiFi.localIP().toString());
  } else {
    digitalWrite(LED_STATUS, LOW);
  }

  tOwm = tDht = tScr = tHistory = millis();
  addHistory();
}

//═══════════════════════════════════════════════════════════════════════════
//  LOOP
//═══════════════════════════════════════════════════════════════════════════
void loop() {
  if (W.wifiOk) server.handleClient();

  unsigned long now = millis();

  // Pulsador manual + rotacion automatica
  static bool btnPrev = HIGH;
  bool btnNow = digitalRead(BTN_PIN);
  if (btnPrev == HIGH && btnNow == LOW) {
    curScreen = (curScreen + 1) % NUM_SCREENS;
    tScr = now;
    lastScreen = 255;
  }
  btnPrev = btnNow;

  if (now - tScr >= SCR_CYCLE) {
    tScr = now;
    curScreen = (curScreen + 1) % NUM_SCREENS;
  }

  // Lectura DHT11 con offset aplicado
  if (now - tDht >= T_DHT) {
    tDht = now;
    float h = dht.readHumidity();
    float t = dht.readTemperature();
    if (!isnan(h) && !isnan(t)) {
      W.hInt  = h;
      W.tInt  = t + cfg_tOffset;   // offset de calibracion
      W.dhtOk = true;
    }
  }
```

```
  // Actualizacion OWM
  if (now - tOwm >= T_OWM && W.wifiOk) {
    tOwm = now;
    fetchOWM();
    fetchForecast();
    lastScreen = 255;
  }

  // Historial
  if (now - tHistory >= T_HISTORY) {
    tHistory = now;
    addHistory();
  }

  // Dibujar solo si cambia la pantalla
  if (curScreen != lastScreen) {
    tft.fillRect(0, BODY_Y, TFT_W, BODY_H, C_BG);
    drawHeader();
    switch (curScreen) {
      case 0: screenClock();    break;
      case 1: screenInterior(); break;
      case 2: screenExterior(); break;
      case 3: screenDetails();  break;
      case 4: screenForecast(); break;
    }
    lastScreen = curScreen;
  }

  delay(100);
}
```

El Código consta de varias secciones, perfectamente encadenadas para que cumpla su objetivo, cómo ya comenté anteriormente, no soy especialista en programación, son varias horas de estudio, pruebas y errores, que se fueron solucionando poco a poco hasta llegar al resultado final.

La sección informativa del código están descritas todas las conexiones físicas entre los distintos componentes, así como los cambios entre las últimas versiones.

En la 1ª sección están declaradas todas las librerías utilizadas, así como las distintas variables y definiciones.

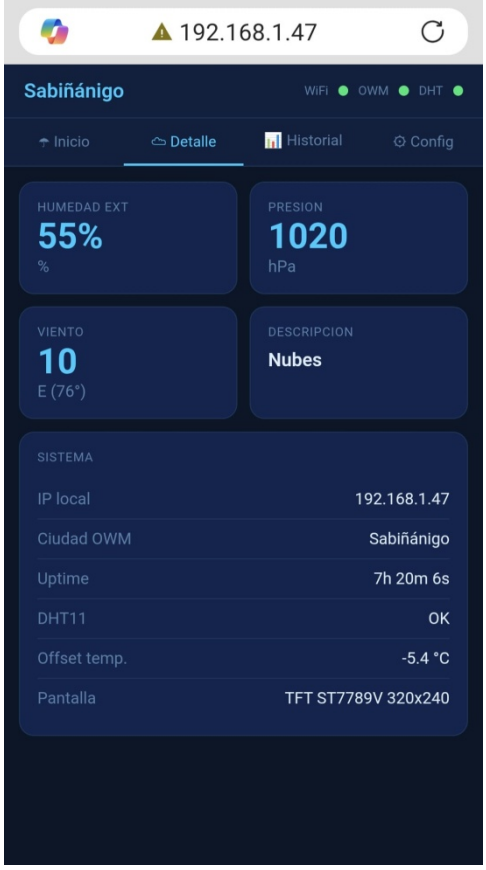
En la “Configuración por Defecto”, pondremos la conexión WIFI, la ciudad y el código internacional del país dónde vivimos, ya que ésta configuración la podemos cambiar en la pantalla “Configuraciones” del Servidor WEB. A continuación están las credenciales para acceder a ésta parte de Configuraciones.

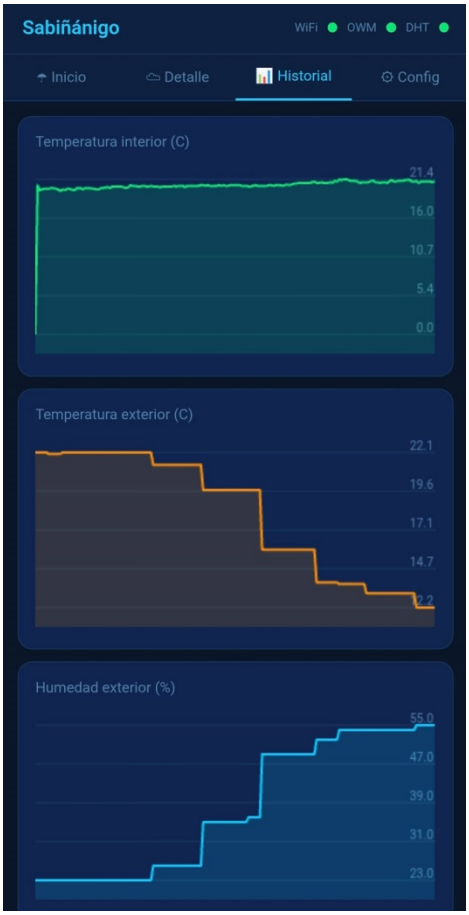
Las otras secciones ya en sí es el desarrollo de todo el código.

Para el acceso por WEB, accedo directamente en el navegador con la IP Local, para acceder desde la misma red wifi y por la IP pública configurada en el Router para acceder desde Internet en cualquier sitio lejos de casa.

En las pantallas del servidor web tenemos la página Inicio, que tiene una cabecera común para todas, que muestra la ciudad y el estado de la conexión wifi, el acceso a internet y el estado del detector DHT11, con unos puntos que si están rellenos en verde es que todo está OK y debajo en forma de tarjetas, nos muestra: Temperatura exterior, Descripción del estado del tiempo (nublado, cielo claro, nevando, etc.) así como la previsión de temperatura máxima y mínima para el día de hoy. Al lado derecho tenemos la Ciudad, la Hora Local y la fecha. En el centro Temperatura y humedad interior y debajo la previsión de los siguientes tres días.

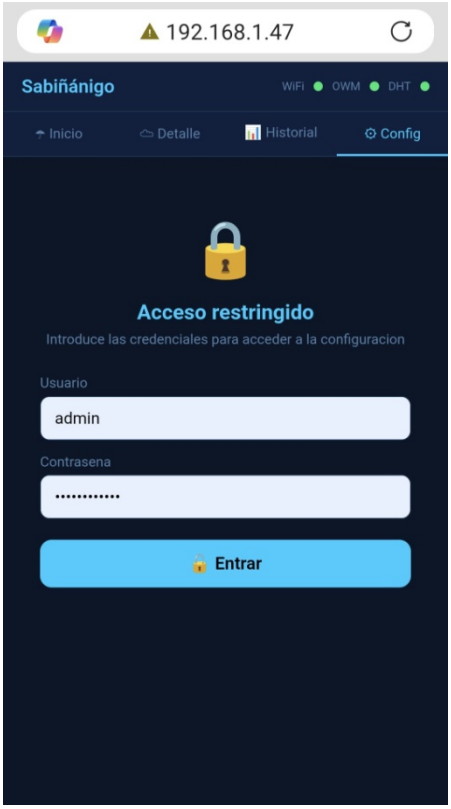
En la pantalla Detalle, también en forma de tarjetas nos muestra la Humedad exterior; La presión atmosférica; El viento: velocidad en Km/h, así como la dirección de donde sopla en cardinales y en grados; La DXescripción del estado del cielo y por íltimo debajo una tarjeta con los datos del Sistema: IP Local, Ciudad, El tiempo de funcionamiento desde el último reinicio, el estado del sensor interior, el ajuste offset del DHT11 y la Pantalla que está utilizando.

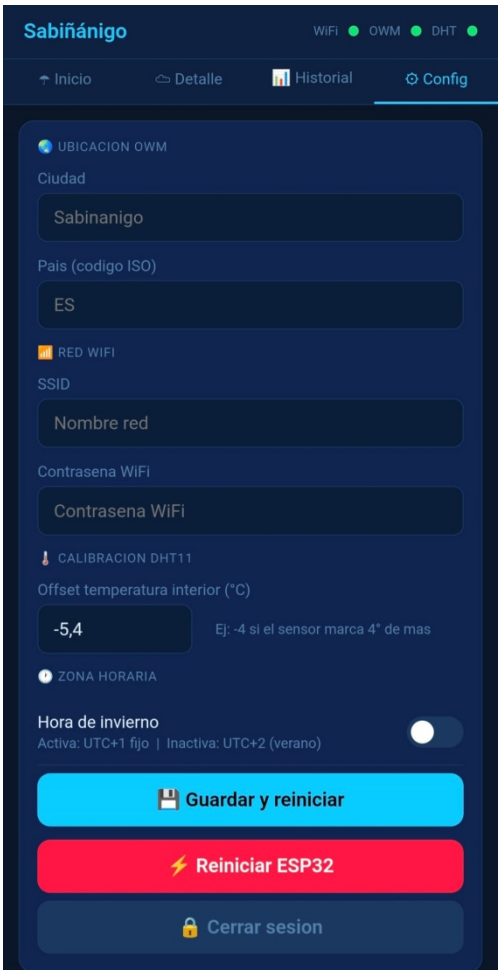




En la pantalla Historial vemos las graficas guardadas en la memoria flahs desde el ultimo reinicio del sistema, registrada según el tiempo de intervalo definido en el código y con un ciclo de 24h. En las que vemos los registros de la Temperatura Interior, Temperatura Exterior, Humedad Exterior y la Presión atmosférica exterior.

A continuación, vemos la pantalla de configuración que para acceder hay que loguearse con el Usuario y Contraseña definido en el código.





En Configuraciones, podemos cambiar los valores de las variables que podemos modificar, cómo son:

La Ciudad.

El Pais.

La Red Wifi (SSID y Contraseña).

El Offset de calibración del DHT11.

Zona Horaria que activa en mi caso la diferencia horaria es de una hora (hora de Verano) y desactivada la diferencia es de dos horas con respecto a la hora UTC .

Espero que les guste el Proyecto y sientánce libre de implementar cualquier modificacíon o mejora a todo el proyecto, me imagino y estoy seguro, que cualquier especialista lo mejora.

Otros proyectos realizados anteriormente entre otros:

- ***Medidor de Potencia y ROE; Arduino NANO, Pantalla OLED, Detector doble de RF y reguladores de voltajes.***
- ***MP3 player; Arduino NANO, Pantalla OLED, MP3 player MINI. USB.***
- ***Estación Meteorológica; Arduino NANO, Pantalla OLED, Módulo WIFI, Termistor.***

Ariel Izquierdo Ereza

2026